create_pg_mesh_pattern

NAME

create_pg_mesh_pattern

Creates a power ground mesh pattern. The mesh pattern can be instantiated in a design by set_pg_strategy command to create power ground mesh.

SYNTAX

```
status create_pg_mesh_pattern
       pattern_name
       -layers layer_expr
       [-parameters var_list]
       [-via_rule via_rule_spec]
```

Data Types

pattern_name	string
layer_expr	specification
var_list	list
via_rule_spec	specification

ARGUMENTS

pattern_name

Specifies the name of the mesh pattern.

-parameters var_list

Specifies the list of parameters. Each parameter can be evaluated and used by other options in this command. This allows the pattern to be programmable and reused by passing different parameters in different situations. Each parameter var in the list var_list is a string and the at character (@) is used as a prefix to evaluate the parameter, such as @var. See the example section for more details.

-layers layer_expr

Specifies the layer configuration for this mesh pattern. Each layer is specified by a layer_expr expression in the following syntax:

```
{horizontal_layer | vertical_layer: layer_name | @var}
{width: minimum | value_list | @var}
{spacing: interleaving | minimum | value_list | @var}
{offset: offset_value | @var}
{pitch: pitch_value | @var}
{trim : true | false | @var}
{track_alignment : track | half_track | auto | @var}
{mask : mask_one | mask_two | mask_three | mask_one_mask_two_alternate |
   mask_two_mask_one_alternate | @var}
{mask_constraint : constraint_name}
```

where the specification contains several keywords, horizontal_layer or vertical_layer, width, offset, pitch, spacing, trim, mask and mask_constraint.

The horizontal_layer or vertical_layer keyword specifies the layer name for this layer. Only one of horizontal_layer and vertical_layer can be specified within one layer. The layer name is specified by layer_name or by evaluating the variable var in -parameters option. The at character (@) is used as a prefix to evaluate the variable. The layer name is a required option in the layer spec.

The width keyword specifies the width of this layer. "minimum" refers to the minimum wire width defined in the technology file. value_list contains a list of width values, where the width value is a float-type value with units defined in the technology file. If the number of width values is smaller than the number of nets from the set_pg_strategy command, the last width value is assumed for the extra nets. The width can also be specified by evaluating the var variable in -parameters option. The at character (@) is used as a prefix to evaluate the variable. By default, the minimum width is used if this option is not specified.

The offset keyword specifies the offset value. offset_value is the offset applied to the center line of the first wire to the boundary of the routing region or a specified absolute coordi-

nate in set_pg_strategy command. By default, the offset is 0. The offset can also be specified by evaluating the var variable in -parameters option.

The pitch keyword specifies the pitch value. pitch_value specifies the pitch of the wires in this layer. The pitch can also be specified by evaluating the var variable in -parameters option. This pitch is a required option in the layer spec.

The spacing keyword specifies the spacing between wires in this layer. minimum refers to the minimum spacing between wires defined in the technology file based on the wire width. value_list contains a list of spacing values, where the spacing value is a float-type value with unit defined in the technology file. Each spacing value specifies the spacing between the adjacent wires in the pattern. For example, the first spacing value specifies the spacing between the first and the second wires, and so on. The number of wires in this pattern is specified by the set_pg_strategy command. If the number of spacing values is smaller than the number of nets minus one, the last spacing value is assumed for the extra nets. interleaving keyword means that distance between adjacent center lines of wires is equal. The spacing can also be specified by evaluating the var variable in -parameters option. The at character (@) character is used as a prefix to evaluate the variable. By default, the minimum spacing is used if this option is not specified.

The trim keyword specifies the trim option for wires in this layer. Valid values are true, false or a value specified by evaluating the var variable in -parameters option. By default, the trim option is true.

The track_alignment keyword specifies the track alignment option for wires in this layer. The option can be track, half_track, or auto. When auto is specified, tool choose the alignment method with smaller number of used tracks. It can also be specified by evaluating the var variable in the -parameters option. The at character (@) character is used as a prefix to evaluate the variable. By default, the wires are created at the specified location and no alignment is performed.

The mask keyword specifies the mask option for wires in this layer. The mask can be specified by one of the keywords, mask_one, mask_two, mask_three, mask_one_mask_two_alternate or mask_two_mask_one_alternate. It can also be specified by evaluating the var variable in the -parameters option. The at character (@) character is used as a prefix to evaluate the variable. By default, the wires are not colored with mask if not specified.

The mask_constraint keyword specifies the mask constraint option for wires in this layer. The mask constraint is defined by the set_pg_mask_constraint command. It can also be specified by evaluating the var variable in the -parameters option. The at character (@) character is used as a prefix to evaluate the variable. By default, the wires are not checked against mask constraint.

Multiple layer expressions can be specified and separated by brace pairs, that is, one layer expression per layer is within a pair of braces.

-via_rule via_rule_spec
Specifies the via_rule_spec via rule between wires in this mesh pattern. The via rule defines the intersection locations and the via at the locations. The rule will only apply between layers of mesh pattern, but won't apply between layer of mesh pattern and layer of other pattern. You must define via rules with the set_pg_strategy_via_rule command if control is needed between layer of mesh pattern and layer of other pattern.

There are several ways to define the intersection locations in the via_rule_spec specification, where the syntax is defined as follows:

{intersection : all} {via_def} |
{intersection : adjacent} {via_def} |
list_of_filter_rules |

The first method is "{intersection : all}", where intersection is the keyword and all refers to all intersections between any two orthogonal wires in any different layers.

The second method is "{intersection : adjacent}", where intersection is the keyword, and adjacent refers to all intersections between any two orthogonal wires only in adjacent layers.

The third method is to define the intersections based on two sets of wire shapes using list_of_filter_rules, which contains a list of filtering rule. Each filtering rule is inside a brace pair. The syntax of each filtering rule is as follows:

{filter_1}{filter_2}{via_def} |
{intersection : undefined}{via_def}

where filter_1 refers to the first filtering operation and filter_2 refer to the second filtering operation. The intersection is defined between the first set and second set of wires based on the filtering rules. The second filtering rule is optional, that is, the intersection is defined based on the first set of wires to all other wires in this mesh.

The filtering operation is defined as follows:

{layers : layer_list} {width : {lower_w upper_w}}

where the layers and width keywords can be used to specify the wire layer names by layer_list or width range by {lower_w upper_w}. Here, layer_list contains a list of layer names, and {lower_w upper_w} is the lower/upper range of wire width. For those intersections which do not belong to any filtering rule, the via definition can be specified as follows:

{intersection : undefined}{via_def}

where intersection is the keyword and undefined refers to the remaining intersections which do not belong to any filtering rules. By default, vias are only created at the intersections which belong to a filtering rule. Multiple via filtering rules for intersection location and via definition can be defined and separated by braces where each filter rule is in one brace pair.

With the intersection definition, the via specification {via_def} uses the following format:

{via_master : via_master_list | via_rule_list}

where via_master is the keyword for via masters including via contact code defined in the technology file or user-defined via cells, The PG via rules defined by set_pg_via_master_rule command can be also specified in the list. via_master_list specifies the list of via masters. Via masters are created at the intersection center without replication. If offset or specific array dimension is needed, via rules specified by set_pg_via_master_rule command can be used. via_rule_list is a list of PG via rules. NIL can be used as a keyword indicate that no via would be created. default can be used to describe the default vias in the specified location. Multiple via rules can be specified and separated by brace pairs. If NIL is not specified in the list, default contact code will be used to complete a stacked via or to replace a via with DRCs when applicable.

By default, the default via defined in the technology file will be created at each intersection of orthogonal wires in different layers.

DESCRIPTION

Creates a power ground mesh pattern. This mesh pattern can be used to create PG mesh with the set_pg_strategy command.

The via rules can also be defined based on intersection or filtering rules. Each via rule contains the intersection location and the via definition by contact code or via master rule.

EXAMPLES

The following example creates a mesh pattern with three layers. Parameters are used for width and offset. The vias between adjacent layers are specified in the via rule. set_pg_strategy and compile_pg are used to create the mesh.

```
prompt> create_pg_mesh_pattern mesh \
   -parameters {w1 w2 w3 f} \
   -layers { \
      {{vertical_layer: M8}{width: @w1} \
```

```
                {spacing: interleaving}{pitch: 20} {offset: @f}} \
            {{horizontal_layer: M7}{width: @w2} \
                {spacing: interleaving} {pitch: 20} {offset: @f}} \
            {{vertical_layer: M6}{width: @w3} \
                {spacing: interleaving}{pitch: 20} {offset: @f}} \
         } \
         -via_rule { \
            {{layers: M8}{layers: M7}{via_master: VIA78}} \
            {{layers: M7}{layers: M6}{via_master: VIA67}} \
         }

      prompt> set_pg_strategy smesh \
         -pattern { \
            {pattern: mesh}{nets: VDD VSS} \
                {parameters: 5 3 1 3}} \
         -core

      prompt> compile_pg

      The following example creates a mesh pattern with layers with same direction.
      Parameters are used for width and offset. The vias between parallel
      layers M8 and M6 are specified in the via rule.

      prompt> create_pg_mesh_pattern mesh \
         -parameters {w1 w3 f} \
         -layers { \
            {{vertical_layer: M8}{width: @w1} \
                {spacing: interleaving}{pitch: 20} {offset: @f}} \
            {{vertical_layer: M6}{width: @w3} \
                {spacing: interleaving}{pitch: 20} {offset: @f}} \
         } \
         -via_rule { \
            {{layers: M8}{layers: M6} \
               {via_master: VIA78 VIA67}{between_parallel: true}} \
         }

   In addition to the above examples, see the  Examples  page  in  the  PG
   Planning  section  of the Task Assistant for more information.  To view
   the Examples page, start the GUI  and  invoke  the  following  command:
   gui_show_task_assistant   -task   "Design  Planning:PG  Planning->Exam-
   ples->Overview".

SEE ALSO
   compile_pg(2)
   create_pg_macro_conn_pattern(2)
   create_pg_ring_pattern(2)
   create_pg_std_cell_conn_pattern(2)
   create_pg_wire_pattern(2)
   remove_pg_patterns(2)
   report_pg_patterns(2)
   report_pg_strategies(2)
   set_pg_strategy(2)
   set_pg_via_master_rule(2)
   set_pg_mask_constraint(2)

                    Version V-2023.12-SP5-6
        Copyright (c) 2025 Synopsys, Inc. All rights reserved.
```
